

DBMS & MySQL

Complete Training Notes

Module 3 — Database Management with MySQL
Python Full Stack Training Program

Topics Covered

- DBMS & RDBMS Concepts
 - MySQL Features
- ER Diagrams & Normalization
- SQL Commands: DDL, DML, DCL, TCL
- Queries: SELECT, WHERE, LIKE, ORDER BY, BETWEEN
 - Joins, Subqueries, Views, Stored Procedures

1. DBMS & RDBMS Concepts

1.1 What is Data, Information and Database?

- **Data:** Raw, unprocessed facts and figures (e.g., 25, "Ravi", "Chennai") that have no meaning by themselves.
- **Information:** Processed data that is meaningful and useful (e.g., "Ravi, age 25, lives in Chennai").
- **Database:** An organized collection of related data stored electronically so that it can be easily accessed, managed, and updated.

1.2 What is a DBMS?

A Database Management System (DBMS) is software that enables users to define, create, maintain, and control access to a database. It acts as an interface between the end user/application and the database, allowing data to be stored, retrieved, and manipulated efficiently and securely.

Examples of DBMS software

- MySQL, Oracle, MongoDB (NoSQL), Microsoft SQL Server, PostgreSQL, SQLite, IBM Db2

Objectives / Advantages of DBMS

- Minimizes data redundancy (duplication) through centralized storage
- Ensures data consistency and integrity across the system
- Provides data security through authentication and authorization
- Supports concurrent access by multiple users with concurrency control
- Provides backup and recovery mechanisms in case of failure
- Enforces data independence — applications are insulated from how data is physically stored
- Supports multiple views of the same data for different users

Disadvantages of DBMS

- High cost of software, hardware, and skilled personnel
- Increased complexity in design and administration
- Requires significant processing overhead for security/backup

1.3 File System vs DBMS

Aspect	Traditional File System	DBMS
Data Redundancy	High — same data duplicated across files	Minimized through normalization and centralization
Data Integrity	Difficult to enforce	Enforced via constraints (PK, FK, checks)
Data Security	Limited, file-level only	Fine-grained, user/role-based access control
Data Sharing	Difficult, files often application-specific	Easy concurrent multi-user sharing
Backup/Recovery	Manual, unreliable	Automated and reliable
Query Capability	Requires custom program per query	Powerful query language (SQL)

Aspect	Traditional File System	DBMS
Data Independence	Not supported — programs tied to file structure	Supported at logical and physical levels

1.4 What is an RDBMS?

A Relational Database Management System (RDBMS) is a type of DBMS that stores data in the form of tables (relations) consisting of rows and columns, based on the Relational Model proposed by E. F. Codd (1970). Relationships between tables are established using keys (Primary Key and Foreign Key).

Key RDBMS Terminology

Term	Meaning
Table / Relation	A collection of related data organized in rows and columns
Tuple / Row / Record	A single entry in a table representing one entity instance
Attribute / Column / Field	A named property of the entity (e.g., Name, Age)
Domain	The set of allowable values for an attribute
Degree	The number of columns/attributes in a table
Cardinality	The number of rows/tuples in a table
Schema	The logical structure/blueprint of the database
Instance	The actual data stored in the database at a given moment

Codd's 12 Rules (Overview)

E. F. Codd defined 13 rules (numbered 0–12) that a database must satisfy to be considered truly relational.

- Rule 0 – Foundation Rule: system must manage data through its relational capabilities
- Rule 1 – Information Rule: all data represented as values in tables
- Rule 2 – Guaranteed Access: every value accessible via table name + primary key + column name
- Rule 3 – Systematic NULL handling for missing/inapplicable data
- Rule 4 – Active online catalog (system catalog describes the database structure)
- Rule 7 – High-level Insert, Update, Delete on relations (set-based operations)
- Rule 10 – Integrity independence (constraints stored in catalog, not application code)

1.5 Types of Database Keys

Key Type	Description
Primary Key	Uniquely identifies each record in a table; cannot be NULL or duplicate
Candidate Key	An attribute (or set) that could qualify as a primary key; a table may have multiple
Alternate Key	A candidate key not chosen as the primary key
Super Key	A set of one or more attributes that uniquely identifies a row (may include extra attributes)
Composite Key	A primary key made up of two or more columns combined

Key Type	Description
Foreign Key	An attribute referencing the primary key of another table; enforces referential integrity
Unique Key	Ensures all values in a column are distinct; unlike PK it can accept one NULL

Code Example — Primary Key, Composite Key, Foreign Key, Unique Key

```
CREATE TABLE students (
  roll_no      INT PRIMARY KEY,
  email        VARCHAR(100) UNIQUE,
  name         VARCHAR(50) NOT NULL
);

CREATE TABLE enrollments (
  roll_no      INT,
  course_id    INT,
  enroll_date  DATE,
  PRIMARY KEY (roll_no, course_id),      -- composite key
  FOREIGN KEY (roll_no) REFERENCES students(roll_no)
);

DESCRIBE enrollments;
```

Output:

```
Query OK, 0 rows affected (0.03 sec)
Query OK, 0 rows affected (0.04 sec)
```

```
+-----+-----+-----+-----+-----+-----+
| Field      | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| roll_no    | int  | NO   | PRI | NULL    |       |
| course_id  | int  | NO   | PRI | NULL    |       |
| enroll_date| date | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

1.6 Data Models

- Hierarchical Model — tree-like parent-child structure
- Network Model — graph structure allowing many-to-many links
- Relational Model — data in tables linked by keys (used by MySQL, Oracle, etc.)
- Object-Oriented Model — data represented as objects (as in OOP)
- NoSQL Models — document, key-value, column-family, graph (e.g., MongoDB)

1.7 Three-Schema Architecture

- External Level (View level) — how individual users see the data
- Conceptual Level (Logical level) — the community/overall logical structure
- Internal Level (Physical level) — how data is actually stored on disk

Note: This separation provides *Data Independence*: *logical independence* (changes at conceptual level don't affect external views) and *physical independence* (changes in storage don't affect the conceptual schema).

1.8 ACID Properties

ACID properties guarantee reliable processing of database transactions:

Property	Meaning
Atomicity	A transaction is all-or-nothing — either fully completes or fully rolls back
Consistency	A transaction brings the database from one valid state to another, preserving all rules/constraints
Isolation	Concurrent transactions execute as though they were run sequentially, without interfering
Durability	Once committed, changes persist permanently even after a system crash

2. MySQL Features

MySQL is one of the most popular open-source Relational Database Management Systems, currently owned and maintained by Oracle Corporation. It uses Structured Query Language (SQL) for accessing and managing data.

2.1 Key Features of MySQL

- **Open Source & Free:** Freely available under the GPL license; also offers commercial editions with extra support
- **Cross-Platform:** Runs on Windows, Linux, macOS, Unix and more
- **High Performance:** Optimized storage engines (InnoDB, MyISAM) deliver fast read/write operations
- **Scalability:** Handles databases from small applications to large-scale, high-traffic systems
- **Security:** Provides user authentication, privilege-based access control, and data encryption (SSL)
- **Multiple Storage Engines:** InnoDB (transactional, supports FK & ACID), MyISAM (fast reads, no transactions), Memory, CSV, Archive, etc.
- **ACID Compliance:** InnoDB engine fully supports ACID transactions
- **Replication Support:** Master-slave and master-master replication for high availability and load balancing
- **Client-Server Architecture:** A central MySQL server handles requests from many client applications
- **Support for Views, Triggers, Stored Procedures:** Enables advanced, reusable, and secure database logic
- **Connector/API Support:** Works with Python (mysql-connector, PyMySQL), Java (JDBC), PHP, Node.js, etc.
- **Data Types Support:** Rich set: numeric, string/text, date/time, JSON, spatial data types
- **Community & Enterprise Editions:** Community edition is free; Enterprise adds backup, monitoring, and support tools

2.2 MySQL Storage Engines Comparison

Feature	InnoDB	MyISAM
Transactions	Supported (ACID compliant)	Not supported
Foreign Keys	Supported	Not supported
Locking	Row-level locking	Table-level locking
Crash Recovery	Automatic recovery	Manual repair needed
Best Use Case	Applications needing data integrity	Read-heavy applications, logging

Code Example — Checking Version and Storage Engines

```
SELECT VERSION();
```

Output:

```
+-----+
| VERSION() |
+-----+
```

```
| 8.0.36 |
+-----+
1 row in set (0.00 sec)
```

```
SHOW ENGINES;
```

Output:

```
+-----+-----+-----+-----+-----+
| Engine | Support | Transactions | XA | Savepoints |
+-----+-----+-----+-----+-----+
| InnoDB | DEFAULT | YES          | YES | YES        |
| MyISAM | YES     | NO           | NO  | NO         |
| MEMORY | YES     | NO           | NO  | NO         |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

2.3 MySQL Architecture (Overview)

- Connection Layer — handles client authentication and connection pooling
- SQL Layer — parses queries, optimizes execution plans, manages caching
- Storage Engine Layer — pluggable engines (InnoDB, MyISAM) manage actual data storage/retrieval

3. ER Diagrams & Normalization

3.1 Entity-Relationship (ER) Model

The ER Model is a conceptual, diagrammatic approach to database design that represents real-world entities and the relationships between them, before translating the design into actual relational tables.

Core Components

Component	Symbol	Description
Entity	Rectangle	A real-world object or concept (e.g., Student, Employee, Product)
Weak Entity	Double Rectangle	An entity that cannot be uniquely identified without a related strong entity
Attribute	Ellipse/Oval	A property or characteristic of an entity (e.g., Name, Age)
Key Attribute	Underlined Ellipse	An attribute that uniquely identifies an entity (e.g., Roll No)
Composite Attribute	Ellipse with sub-ellipses	An attribute divisible into smaller parts (e.g., Address → Street, City)
Multivalued Attribute	Double Ellipse	An attribute that can hold multiple values (e.g., Phone Numbers)
Derived Attribute	Dashed Ellipse	A value derived from another attribute (e.g., Age from Date of Birth)
Relationship	Diamond	An association between two or more entities (e.g., Enrolls, Manages)
Link/Line	Straight Line	Connects entities to attributes and to relationships

Types of Attributes — Quick Recap

- Simple Attribute — cannot be divided further (e.g., Age)
- Composite Attribute — can be broken into sub-parts (e.g., Name → First, Last)
- Single-valued Attribute — holds only one value (e.g., DOB)
- Multi-valued Attribute — can hold several values (e.g., Email IDs)
- Derived Attribute — computed from other attributes (e.g., Age from DOB)

Degree of a Relationship

- Unary (Degree 1) — relationship of an entity with itself (e.g., Employee manages Employee)
- Binary (Degree 2) — relationship between two entities (most common, e.g., Student enrolls Course)
- Ternary (Degree 3) — relationship among three entities (e.g., Supplier supplies Part to Project)

Cardinality (Relationship Types)

Cardinality	Description	Example
One-to-One (1:1)	One entity instance relates to exactly one instance of another	A Person has one Passport

Cardinality	Description	Example
One-to-Many (1:N)	One entity instance relates to many instances of another	A Department has many Employees
Many-to-One (N:1)	Many instances relate to a single instance (inverse of 1:N)	Many Students belong to one Class
Many-to-Many (M:N)	Many instances relate to many instances of another	Students enroll in many Courses

Participation Constraints

- Total Participation (double line) — every entity instance must participate in the relationship
- Partial Participation (single line) — participation in the relationship is optional

3.2 Converting ER Diagrams to Tables (Mapping Rules)

- Each strong entity becomes a table; its attributes become columns; key attribute becomes the Primary Key
- Each weak entity becomes a table whose primary key includes the primary key of its owner entity (as a foreign key) plus its own partial key
- A 1:1 relationship can be merged into one of the two tables (foreign key placed on either side)
- A 1:N relationship — place the foreign key (referencing the '1' side) into the table on the 'N' side
- An M:N relationship — requires a separate junction/bridge table containing foreign keys from both entities
- Multivalued attributes become a separate table linked by a foreign key

3.3 Normalization

Normalization is the systematic process of organizing columns and tables in a relational database to minimize data redundancy and eliminate undesirable characteristics like Insertion, Update, and Deletion anomalies.

Anomalies Addressed by Normalization

- **Insertion Anomaly:** Unable to add data unless unrelated/unwanted data is also added (e.g., cannot add a course without a student enrolled).
- **Update Anomaly:** Same data repeated in multiple rows means an update must be made in many places, risking inconsistency.
- **Deletion Anomaly:** Deleting a record unintentionally removes other useful data (e.g., deleting the last student of a course deletes course info too).

Functional Dependency

A functional dependency $X \rightarrow Y$ means that the value of attribute X uniquely determines the value of attribute Y. This concept underlies all normal forms.

Normal Forms

Normal Form	Rule	Removes
1NF (First Normal Form)	Each column holds atomic (indivisible) values; no repeating groups or multivalued columns	Repeating groups

Normal Form	Rule	Removes
2NF (Second Normal Form)	Must be in 1NF; every non-key attribute must fully depend on the entire (composite) primary key	Partial dependency
3NF (Third Normal Form)	Must be in 2NF; no transitive dependency — non-key attributes must not depend on other non-key attributes	Transitive dependency
BCNF (Boyce-Codd Normal Form)	A stricter version of 3NF; for every functional dependency $X \rightarrow Y$, X must be a super key	Anomalies from overlapping candidate keys
4NF (Fourth Normal Form)	Must be in BCNF; no multi-valued dependency	Multi-valued dependency
5NF (Fifth Normal Form)	Must be in 4NF; no join dependency — table cannot be decomposed further without loss	Join dependency

Worked Example — Normalizing a Table

Unnormalized table (Student, Courses, CourseInstructor) with a multivalued Courses column:

StudentID	StudentName	Courses	Instructor
101	Arun	Maths, Physics	Mr. Iyer, Ms. Rao
102	Divya	Chemistry	Dr. Nair

Step 1 — Convert to 1NF (atomic values, one course per row):

StudentID	StudentName	Course	Instructor
101	Arun	Maths	Mr. Iyer
101	Arun	Physics	Ms. Rao
102	Divya	Chemistry	Dr. Nair

Step 2 — 2NF: separate Student info (depends only on StudentID) from Course/Instructor info (depends on Course):

```
Students: StudentID, StudentName
Enrollments: StudentID, Course
Courses: Course, Instructor
```

Step 3 — 3NF: ensure Instructor depends only on Course, not transitively through another non-key attribute (already satisfied above since $\text{Course} \rightarrow \text{Instructor}$ directly).

Note: In real training/practice, always identify the primary key and all functional dependencies first — normalization rules follow logically from these dependencies.

4. SQL Commands: DDL, DML, DCL, TCL

SQL (Structured Query Language) commands are grouped into five categories based on their function:

Category	Full Form	Purpose	Commands
DDL	Data Definition Language	Defines/modifies database structure (schema)	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	Manipulates the actual data within tables	INSERT, UPDATE, DELETE, (SELECT is sometimes classed as DQL)
DQL	Data Query Language	Retrieves data from the database	SELECT
DCL	Data Control Language	Controls access/permissions on data	GRANT, REVOKE
TCL	Transaction Control Language	Manages transactions within the database	COMMIT, ROLLBACK, SAVEPOINT, SET TRANSACTION

4.1 DDL — Data Definition Language

DDL commands define and modify the structure of database objects (databases, tables, indexes). DDL statements in MySQL are auto-committed — they cannot be rolled back.

CREATE

Creates a new database, table, index, or view.

```
CREATE DATABASE company;

USE company;

CREATE TABLE employees (
    emp_id      INT PRIMARY KEY AUTO_INCREMENT,
    emp_name    VARCHAR(50) NOT NULL,
    department  VARCHAR(30),
    salary      DECIMAL(10,2) DEFAULT 0.00,
    hire_date   DATE
);
```

Output:

```
Query OK, 1 row affected (0.02 sec)

Database changed

Query OK, 0 rows affected (0.04 sec)
```

ALTER

Modifies the structure of an existing table — add, modify, or drop columns/constraints.

```
ALTER TABLE employees ADD COLUMN email VARCHAR(100);
```

Output:

```
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12,2);
```

Output:

```
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
ALTER TABLE employees DROP COLUMN email;
```

Output:

```
Query OK, 0 rows affected (0.03 sec)
```

```
ALTER TABLE employees RENAME COLUMN department TO dept;
```

Output:

```
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
ALTER TABLE employees RENAME COLUMN dept TO department; -- reverted for later examples
ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary >= 0);
```

Output:

```
Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.03 sec)
```

DROP

Permanently deletes an entire table, database, or index along with its structure and data.

```
DROP TABLE employees;
DROP DATABASE company;
```

Output:

```
Query OK, 0 rows affected (0.01 sec)
Query OK, 3 rows affected (0.02 sec)
```

Note: The examples above are shown for syntax reference only. The employees table is assumed to be re-created (as shown under CREATE) for all examples that follow.

TRUNCATE

Removes all rows from a table quickly, resetting AUTO_INCREMENT, but keeps the table structure intact. It is faster than DELETE because it does not log individual row deletions.

```
TRUNCATE TABLE employees;
```

Output:

```
Query OK, 0 rows affected (0.01 sec)
```

RENAME

```
RENAME TABLE employees TO staff;
RENAME TABLE staff TO employees; -- reverted for later examples
```

Output:

```
Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.01 sec)
```

DDL: DROP vs TRUNCATE vs DELETE

Aspect	DROP	TRUNCATE	DELETE
Type	DDL	DDL	DML
Removes	Entire table structure + data	All rows, keeps structure	Rows (all or filtered by WHERE)
WHERE clause	Not applicable	Not applicable	Supported
Rollback	Cannot be rolled back	Cannot be rolled back (in MySQL)	Can be rolled back (within transaction)
Triggers fired	No	No	Yes
Speed	Fast	Very fast	Slower for large data

4.2 DML — Data Manipulation Language

DML commands are used to insert, update, and delete data stored in tables.

INSERT

```
INSERT INTO employees (emp_name, department, salary, hire_date)
VALUES ('Ravi Kumar', 'IT', 55000.00, '2023-06-15');
```

Output:

```
Query OK, 1 row affected (0.01 sec)
```

```
-- Multiple rows at once
INSERT INTO employees (emp_name, department, salary, hire_date) VALUES
('Anita Sharma', 'HR', 48000.00, '2022-01-10'),
('Vikram Singh', 'Finance', 62000.00, '2021-09-01');
```

Output:

```
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
SELECT * FROM employees;
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 1      | Ravi Kumar | IT        | 55000.00 | 2023-06-15 |
| 2      | Anita Sharma | HR        | 48000.00 | 2022-01-10 |
| 3      | Vikram Singh | Finance   | 62000.00 | 2021-09-01 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

UPDATE

```
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'IT';
```

Output:

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
SELECT emp_name, salary FROM employees WHERE department = 'IT';
```

Output:

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Ravi Kumar | 60500.00 |
+-----+-----+
1 row in set (0.00 sec)
```

Note: Always use a *WHERE* clause with *UPDATE* and *DELETE* — omitting it affects every row in the table.

DELETE

```
DELETE FROM employees
WHERE emp_id = 3;
```

Output:

```
Query OK, 1 row affected (0.00 sec)
```

```
SELECT * FROM employees;
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 1      | Ravi Kumar | IT          | 60500.00 | 2023-06-15 |
| 2      | Anita Sharma | HR          | 48000.00 | 2022-01-10 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

4.3 DCL — Data Control Language

DCL commands manage user permissions and access rights on database objects.

GRANT

Gives specific privileges to a user.

```
GRANT SELECT, INSERT, UPDATE ON company.employees TO 'naveen'@'localhost';
```

Output:

```
Query OK, 0 rows affected (0.02 sec)
```

```
SHOW GRANTS FOR 'naveen'@'localhost';
```

Output:

```
+-----+-----+
| Grants for naveen@localhost |
+-----+-----+
| GRANT USAGE ON *.* TO `naveen`@`localhost` |
| GRANT SELECT, INSERT, UPDATE ON `company`.`employees` TO `naveen`@`localhost` |
+-----+-----+
2 rows in set (0.00 sec)
```

REVOKE

Removes previously granted privileges from a user.

```
REVOKE INSERT, UPDATE ON company.employees FROM 'naveen'@'localhost';
```

Output:

```
Query OK, 0 rows affected (0.01 sec)
```

Common Privilege Types

- SELECT, INSERT, UPDATE, DELETE — data-level privileges
- CREATE, ALTER, DROP, INDEX — schema-level privileges
- ALL PRIVILEGES — grants every available privilege
- GRANT OPTION — allows a user to grant their own privileges to others

4.4 TCL — Transaction Control Language

TCL commands manage transactions — a transaction is a logical unit of work made up of one or more SQL statements executed as a whole (see ACID properties, Section 1.8).

COMMIT

Permanently saves all changes made in the current transaction.

```
START TRANSACTION;
UPDATE employees SET salary = salary + 5000 WHERE emp_id = 1;
COMMIT;
```

Output:

```
Query OK, 0 rows affected (0.00 sec)
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0
Query OK, 0 rows affected (0.01 sec)
```

ROLLBACK

Undoes changes made in the current transaction, reverting to the last committed state.

```
START TRANSACTION;
DELETE FROM employees WHERE department = 'HR';
ROLLBACK; -- deletion is undone
```

Output:

```
Query OK, 0 rows affected (0.00 sec)
Query OK, 1 row affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
```

```
SELECT * FROM employees WHERE department = 'HR'; -- row still present after rollback
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 2      | Anita Sharma | HR          | 48000.00 | 2022-01-10 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

SAVEPOINT

Sets a named point within a transaction to which you can later roll back, without undoing the entire transaction.

```
START TRANSACTION;
UPDATE employees SET salary = 60000 WHERE emp_id = 2;
SAVEPOINT sp1;
UPDATE employees SET salary = 70000 WHERE emp_id = 1;
ROLLBACK TO sp1;  -- undoes only the second update
COMMIT;
```

Output:

```
Query OK, 0 rows affected (0.00 sec)
Query OK, 1 row affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 1 row affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
```

```
SELECT emp_id, salary FROM employees WHERE emp_id IN (1,2);
```

Output:

```
+-----+-----+
| emp_id | salary |
+-----+-----+
| 1      | 65500.00 |
| 2      | 60000.00 |
+-----+-----+
2 rows in set (0.00 sec)
```

Note: *emp_id 1 kept its pre-savepoint value (rolled back), while emp_id 2's update — made before the savepoint — was preserved.*

SET TRANSACTION

Sets the isolation level for the current transaction (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE).

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT @@transaction_isolation;
```

Output:

```
Query OK, 0 rows affected (0.00 sec)

+-----+
| @@transaction_isolation |
+-----+
| READ-COMMITTED          |
+-----+
1 row in set (0.00 sec)
```

5. Queries: SELECT, WHERE, LIKE, ORDER BY, BETWEEN

All examples in this section use the following sample employees table (note row 6 has a NULL department, included deliberately to demonstrate NULL-handling):

emp_id	emp_name	department	salary	hire_date
1	Arun Kumar	IT	55000.00	2023-06-15
2	Anita Sharma	HR	48000.00	2022-01-10
3	Vikram Singh	Finance	62000.00	2021-09-01
4	Ravi Iyer	IT	51000.00	2022-03-20
5	Divya Nair	IT	47000.00	2023-01-05
6	Karthik Raj	NULL	45000.00	2023-02-11

5.1 SELECT Statement

SELECT retrieves data from one or more tables. It is the most frequently used SQL command.

```
SELECT * FROM employees;
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 1      | Arun Kumar | IT        | 55000.00 | 2023-06-15 |
| 2      | Anita Sharma | HR        | 48000.00 | 2022-01-10 |
| 3      | Vikram Singh | Finance   | 62000.00 | 2021-09-01 |
| 4      | Ravi Iyer   | IT        | 51000.00 | 2022-03-20 |
| 5      | Divya Nair  | IT        | 47000.00 | 2023-01-05 |
| 6      | Karthik Raj | NULL      | 45000.00 | 2023-02-11 |
+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

```
SELECT emp_name, salary FROM employees;
```

Output:

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Arun Kumar | 55000.00 |
| Anita Sharma | 48000.00 |
| Vikram Singh | 62000.00 |
| Ravi Iyer   | 51000.00 |
| Divya Nair  | 47000.00 |
| Karthik Raj | 45000.00 |
+-----+-----+
6 rows in set (0.00 sec)
```

```
SELECT DISTINCT department FROM employees;
```

Output:

```
+-----+
| department |
+-----+
```

```

| IT      |
| HR      |
| Finance |
| NULL    |
+-----+
4 rows in set (0.00 sec)

```

General Order of a SELECT Query

```

SELECT column_list
FROM table_name
WHERE condition
GROUP BY column
HAVING group_condition
ORDER BY column [ASC|DESC]
LIMIT n;

```

Note: Logical execution order differs from written order: *FROM* → *WHERE* → *GROUP BY* → *HAVING* → *SELECT* → *ORDER BY* → *LIMIT*.

5.2 WHERE Clause

Filters rows based on a specified condition, applied before grouping.

```
SELECT * FROM employees WHERE department = 'IT';
```

Output:

```

+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 1      | Arun Kumar | IT      | 55000.00 | 2023-06-15 |
| 4      | Ravi Iyer  | IT      | 51000.00 | 2022-03-20 |
| 5      | Divya Nair | IT      | 47000.00 | 2023-01-05 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

```
SELECT * FROM employees WHERE salary > 50000;
```

Output:

```

+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 1      | Arun Kumar | IT      | 55000.00 | 2023-06-15 |
| 3      | Vikram Singh | Finance | 62000.00 | 2021-09-01 |
| 4      | Ravi Iyer  | IT      | 51000.00 | 2022-03-20 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

```
SELECT * FROM employees WHERE department = 'IT' AND salary > 50000;
```

Output:

```

+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 1      | Arun Kumar | IT      | 55000.00 | 2023-06-15 |
| 4      | Ravi Iyer  | IT      | 51000.00 | 2022-03-20 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

```
SELECT * FROM employees WHERE department = 'HR' OR department = 'Finance';
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 2      | Anita Sharma | HR      | 48000.00 | 2022-01-10 |
| 3      | Vikram Singh | Finance | 62000.00 | 2021-09-01 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
SELECT * FROM employees WHERE NOT department = 'IT';
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 2      | Anita Sharma | HR      | 48000.00 | 2022-01-10 |
| 3      | Vikram Singh | Finance | 62000.00 | 2021-09-01 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Note: Karthik Raj (department = NULL) does not appear in either the '=' or 'NOT' result — comparisons against NULL evaluate to UNKNOWN, not TRUE or FALSE, in SQL's three-valued logic.

Comparison & Logical Operators used with WHERE

Operator	Meaning
=, !=, <>	Equal to / Not equal to
<, >, <=, >=	Less than, greater than (and equal to)
AND, OR, NOT	Logical combination of conditions
IN (list)	Matches any value in a given list
BETWEEN a AND b	Matches values within an inclusive range
LIKE pattern	Pattern matching for strings
IS NULL / IS NOT NULL	Checks for NULL values

IN Operator Example

```
SELECT * FROM employees WHERE department IN ('HR', 'Finance');
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 2      | Anita Sharma | HR      | 48000.00 | 2022-01-10 |
| 3      | Vikram Singh | Finance | 62000.00 | 2021-09-01 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

IS NULL Example

```
SELECT * FROM employees WHERE department IS NULL;
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 6      | Karthik Raj | NULL      | 45000.00 | 2023-02-11 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

5.3 LIKE Operator (Pattern Matching)

LIKE is used with WHERE to search for a specified pattern within a column, using wildcard characters.

Wildcard	Meaning	Example
%	Matches zero or more characters	'A%' matches names starting with A
_	Matches exactly one character	'_avi%' matches 2nd-4th letters 'avi'

```
SELECT * FROM employees WHERE emp_name LIKE 'A%';    -- starts with A
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 1      | Arun Kumar | IT        | 55000.00 | 2023-06-15 |
| 2      | Anita Sharma | HR        | 48000.00 | 2022-01-10 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
SELECT * FROM employees WHERE emp_name LIKE '%a';    -- ends with a
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 2      | Anita Sharma | HR        | 48000.00 | 2022-01-10 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```
SELECT * FROM employees WHERE emp_name LIKE '%an%';  -- contains 'an'
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 2      | Anita Sharma | HR        | 48000.00 | 2022-01-10 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```
SELECT * FROM employees WHERE emp_name LIKE '_avi%'; -- 2nd-4th letters 'avi'
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 4      | Ravi Iyer | IT        | 51000.00 | 2022-03-20 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```
SELECT * FROM employees WHERE emp_name NOT LIKE 'A%'; -- does not start with A
```

Output:

```
+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary | hire_date |
+-----+-----+-----+-----+-----+
| 3      | Vikram Singh | Finance | 62000.00 | 2021-09-01 |
| 4      | Ravi Iyer | IT | 51000.00 | 2022-03-20 |
| 5      | Divya Nair | IT | 47000.00 | 2023-01-05 |
| 6      | Karthik Raj | NULL | 45000.00 | 2023-02-11 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

5.4 ORDER BY Clause

Sorts the result set by one or more columns, in ascending (ASC, default) or descending (DESC) order.

```
SELECT emp_name, salary FROM employees ORDER BY salary; -- ascending (default)
```

Output:

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Karthik Raj | 45000.00 |
| Divya Nair | 47000.00 |
| Anita Sharma | 48000.00 |
| Ravi Iyer | 51000.00 |
| Arun Kumar | 55000.00 |
| Vikram Singh | 62000.00 |
+-----+-----+
6 rows in set (0.00 sec)
```

```
SELECT emp_name, salary FROM employees ORDER BY salary DESC;
```

Output:

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Vikram Singh | 62000.00 |
| Arun Kumar | 55000.00 |
| Ravi Iyer | 51000.00 |
| Anita Sharma | 48000.00 |
| Divya Nair | 47000.00 |
| Karthik Raj | 45000.00 |
+-----+-----+
6 rows in set (0.00 sec)
```

```
SELECT emp_name, department, salary FROM employees
ORDER BY department ASC, salary DESC; -- multi-column sort
```

Output:

```
+-----+-----+-----+
| emp_name | department | salary |
+-----+-----+-----+
| Karthik Raj | NULL | 45000.00 |
| Vikram Singh | Finance | 62000.00 |
| Anita Sharma | HR | 48000.00 |
| Arun Kumar | IT | 55000.00 |
| Ravi Iyer | IT | 51000.00 |
+-----+-----+-----+
```

```
| Divya Nair | IT | 47000.00 |
+-----+-----+
6 rows in set (0.00 sec)
```

Note: MySQL sorts NULL values first when using ASC order, which is why Karthik Raj's row appears at the top.

5.5 BETWEEN Operator

Selects values within a given inclusive range — works with numbers, text, and dates.

```
SELECT emp_name, salary FROM employees WHERE salary BETWEEN 40000 AND 60000;
```

Output:

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Karthik Raj | 45000.00 |
| Divya Nair | 47000.00 |
| Anita Sharma | 48000.00 |
| Ravi Iyer | 51000.00 |
| Arun Kumar | 55000.00 |
+-----+-----+
5 rows in set (0.00 sec)
```

```
SELECT emp_name, hire_date FROM employees
WHERE hire_date BETWEEN '2022-01-01' AND '2023-12-31';
```

Output:

```
+-----+-----+
| emp_name | hire_date |
+-----+-----+
| Anita Sharma | 2022-01-10 |
| Ravi Iyer | 2022-03-20 |
| Divya Nair | 2023-01-05 |
| Karthik Raj | 2023-02-11 |
| Arun Kumar | 2023-06-15 |
+-----+-----+
5 rows in set (0.00 sec)
```

```
SELECT emp_name, salary FROM employees WHERE salary NOT BETWEEN 40000 AND 60000;
```

Output:

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Vikram Singh | 62000.00 |
+-----+-----+
1 row in set (0.00 sec)
```

5.6 Other Frequently Used Clauses

GROUP BY and HAVING

```
SELECT department, COUNT(*) AS total_emp, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

Output:

```

+-----+-----+-----+
| department | total_emp | avg_salary |
+-----+-----+-----+
| IT          | 3         | 51000.000000 |
| Finance     | 1         | 62000.000000 |
+-----+-----+-----+
2 rows in set (0.00 sec)

```

Note: *WHERE filters rows before grouping; HAVING filters groups after aggregation. HR (avg 48000) and the NULL-department group (avg 45000) are excluded because they don't satisfy HAVING AVG(salary) > 50000.*

LIMIT and OFFSET

```
SELECT emp_name, salary FROM employees ORDER BY salary DESC LIMIT 5;  -- top 5
```

Output:

```

+-----+-----+
| emp_name | salary |
+-----+-----+
| Vikram Singh | 62000.00 |
| Arun Kumar   | 55000.00 |
| Ravi Iyer    | 51000.00 |
| Anita Sharma | 48000.00 |
| Divya Nair   | 47000.00 |
+-----+-----+
5 rows in set (0.00 sec)

```

```
SELECT emp_name, salary FROM employees ORDER BY salary DESC LIMIT 5 OFFSET 10;
```

Output:

```
Empty set (0.00 sec)
```

Note: *Only 6 rows exist in the table, so skipping the first 10 (OFFSET 10) leaves nothing to return.*

Aggregate Functions

Function	Purpose
COUNT()	Counts number of rows
SUM()	Adds up numeric values
AVG()	Calculates average
MIN() / MAX()	Finds smallest / largest value

```
SELECT COUNT(*) AS total, SUM(salary) AS total_salary,
       AVG(salary) AS avg_salary, MIN(salary) AS min_salary, MAX(salary) AS max_salary
FROM employees;
```

Output:

```

+-----+-----+-----+-----+-----+
| total | total_salary | avg_salary | min_salary | max_salary |
+-----+-----+-----+-----+-----+
| 6     | 308000.00    | 51333.333333 | 45000.00   | 62000.00   |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

```


6. Joins, Subqueries, Views, Stored Procedures

6.1 Joins

A JOIN combines rows from two or more tables based on a related column, typically a primary key–foreign key relationship.

Sample tables used in every example below:

emp_id	emp_name	dept_id	manager_id
1	Arun	1	NULL
2	Anita	2	1
3	Vikram	3	1
4	Ravi	1	1
5	Divya	NULL	4

dept_id	dept_name
1	IT
2	HR
3	Finance
4	Marketing

INNER JOIN

Returns only rows that have matching values in both tables.

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Output:

```
+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Arun      | IT        |
| Anita     | HR        |
| Vikram    | Finance   |
| Ravi      | IT        |
+-----+-----+
4 rows in set (0.00 sec)
```

Note: Divya is excluded because her dept_id is NULL and Marketing is excluded because no employee has dept_id = 4.

LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table, and matched rows from the right table; unmatched right-side columns appear as NULL.

```
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

Output:

```

+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Arun      | IT        |
| Anita     | HR        |
| Vikram    | Finance   |
| Ravi      | IT        |
| Divya     | NULL      |
+-----+-----+
5 rows in set (0.00 sec)

```

RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table, and matched rows from the left table; unmatched left-side columns appear as NULL.

```

SELECT e.emp_name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;

```

Output:

```

+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Arun      | IT        |
| Ravi      | IT        |
| Anita     | HR        |
| Vikram    | Finance   |
| NULL      | Marketing |
+-----+-----+
5 rows in set (0.00 sec)

```

FULL OUTER JOIN

Returns all rows when there is a match in either table. MySQL has no native FULL OUTER JOIN — it is simulated using UNION of LEFT and RIGHT JOIN.

```

SELECT e.emp_name, d.dept_name FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;

```

Output:

```

+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Arun      | IT        |
| Anita     | HR        |
| Vikram    | Finance   |
| Ravi      | IT        |
| Divya     | NULL      |
| NULL      | Marketing |
+-----+-----+
6 rows in set (0.00 sec)

```

SELF JOIN

Joins a table to itself, typically to compare rows within the same table (e.g., employee-manager relationships).

```
SELECT a.emp_name AS Employee, b.emp_name AS Manager
FROM employees a
JOIN employees b ON a.manager_id = b.emp_id;
```

Output:

```
+-----+-----+
| Employee | Manager |
+-----+-----+
| Anita    | Arun    |
| Vikram   | Arun    |
| Ravi     | Arun    |
| Divya    | Ravi    |
+-----+-----+
4 rows in set (0.00 sec)
```

Note: Arun has no manager (*manager_id* is NULL), so he does not appear in the Employee column of this INNER-style self join.

CROSS JOIN

Returns the Cartesian product — every row of the first table combined with every row of the second.

```
SELECT e.emp_name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

Output:

```
+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Arun     | IT        |
| Arun     | HR        |
| Arun     | Finance   |
| Arun     | Marketing |
| Anita    | IT        |
| Anita    | HR        |
| ...      | ...       |
+-----+-----+
20 rows in set (0.00 sec) -- 5 employees x 4 departments
```

Joins — Visual Summary

Join Type	Returns
INNER JOIN	Only matching rows in both tables
LEFT JOIN	All left rows + matched right rows (NULL if no match)
RIGHT JOIN	All right rows + matched left rows (NULL if no match)
FULL OUTER JOIN	All rows from both tables (matched + unmatched)
SELF JOIN	Table joined with itself
CROSS JOIN	Cartesian product of both tables

6.2 Subqueries (Nested Queries)

A subquery is a query nested inside another SQL query (SELECT, INSERT, UPDATE, or DELETE), enclosed in parentheses. It is evaluated first, and its result is used by the outer query.

These examples add a salary column to the employees table used above and reuse the same data (Arun 55000, Anita 48000, Vikram 62000, Ravi 51000, Divya 47000):

Single-row Subquery

```
SELECT emp_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Output:

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Arun      | 55000.00 |
| Vikram    | 62000.00 |
+-----+-----+
2 rows in set (0.00 sec)
```

Note: The average salary across all 5 employees is 52600.00, so only Arun and Vikram exceed it.

Multi-row Subquery (with IN)

```
SELECT emp_name
FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE dept_name = 'IT');
```

Output:

```
+-----+
| emp_name |
+-----+
| Arun      |
| Ravi      |
+-----+
2 rows in set (0.00 sec)
```

Correlated Subquery

References a column from the outer query, so it is re-evaluated once per row of the outer query.

```
SELECT emp_name, salary, dept_id
FROM employees e1
WHERE salary > (
    SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e1.dept_id
);
```

Output:

```
+-----+-----+-----+
| emp_name | salary | dept_id |
+-----+-----+-----+
| Arun      | 55000.00 | 1       |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Note: IT department average is $(55000+51000)/2 = 53000$; only Arun (55000) exceeds his own department's average.

Subquery in FROM Clause (Derived Table)

```
SELECT dept_id, avg_sal
FROM (SELECT dept_id, AVG(salary) AS avg_sal FROM employees GROUP BY dept_id) AS
dept_avg
WHERE avg_sal > 50000;
```

Output:

```
+-----+-----+
| dept_id | avg_sal |
+-----+-----+
| 1       | 53000.000000 |
| 3       | 62000.000000 |
+-----+-----+
2 rows in set (0.00 sec)
```

EXISTS / NOT EXISTS

```
SELECT dept_name FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

Output:

```
+-----+
| dept_name |
+-----+
| IT        |
| HR        |
| Finance   |
+-----+
3 rows in set (0.00 sec)
```

Note: Marketing (dept_id 4) has no matching employee row, so EXISTS returns false for it and it is excluded.

Subquery vs Join

Aspect	Subquery	Join
Purpose	Often used for filtering based on aggregated/derived value	Used to combine columns from multiple tables
Readability	Can be simpler to read for conditional logic	Better for retrieving combined column data
Performance	Can be slower, especially correlated subqueries	Generally optimized well by the query engine

6.3 Views

A View is a virtual table based on the result of a stored SQL SELECT query. It does not store data itself (in most cases) but presents data from one or more underlying tables.

Creating and Using a View

```
CREATE VIEW it_employees AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE dept_id = (SELECT dept_id FROM departments WHERE dept_name = 'IT');
```

Output:

```
Query OK, 0 rows affected (0.02 sec)
```

```
SELECT * FROM it_employees; -- query the view like a table
```

Output:

```
+-----+-----+-----+
| emp_id | emp_name | salary |
+-----+-----+-----+
| 1      | Arun     | 55000.00 |
| 4      | Ravi     | 51000.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Updating and Dropping a View

```
CREATE OR REPLACE VIEW it_employees AS
SELECT emp_id, emp_name, salary, manager_id
FROM employees WHERE dept_id = 1;
```

Output:

```
Query OK, 0 rows affected (0.02 sec)
```

```
DROP VIEW it_employees;
```

Output:

```
Query OK, 0 rows affected (0.01 sec)
```

Advantages of Views

- Simplifies complex queries by hiding join/filter logic behind a simple SELECT
- Provides an additional security layer — users can be given access to a view without access to underlying tables
- Provides logical data independence from changes in the base table structure
- Allows presenting customized/restricted subsets of data to different users

Limitations of Views

- Views based on complex queries (joins, GROUP BY, aggregate functions) are often not updatable
- Can introduce a slight performance overhead since the underlying query re-executes

6.4 Stored Procedures

A Stored Procedure is a precompiled collection of one or more SQL statements stored in the database, which can be executed (called) as a single unit — similar to a function/method in programming.

These examples use a simple employees table: (1, 'Ravi Kumar', 'IT', 45000.00), (2, 'Anita Sharma', 'IT', 58000.00), (3, 'Vikram Singh', 'HR', 52000.00).

Creating a Simple Stored Procedure

```
DELIMITER //

CREATE PROCEDURE GetEmployeesByDept(IN dept VARCHAR(30))
BEGIN
    SELECT emp_name, salary
    FROM employees
    WHERE department = dept;
END //

DELIMITER ;
```

Output:

```
Query OK, 0 rows affected (0.01 sec)
```

```
-- Calling the procedure
CALL GetEmployeesByDept('IT');
```

Output:

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Ravi Kumar | 45000.00 |
| Anita Sharma | 58000.00 |
+-----+-----+
2 rows in set (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
```

Stored Procedure with IN, OUT Parameters

```
DELIMITER //

CREATE PROCEDURE GetAvgSalary(IN dept VARCHAR(30), OUT avg_sal DECIMAL(10,2))
BEGIN
    SELECT AVG(salary) INTO avg_sal
    FROM employees
    WHERE department = dept;
END //

DELIMITER ;
```

Output:

```
Query OK, 0 rows affected (0.01 sec)
```

```
CALL GetAvgSalary('IT', @result);
SELECT @result;
```

Output:

```
Query OK, 1 row affected (0.00 sec)

+-----+
| @result |
+-----+
| 51500.00 |
+-----+
1 row in set (0.00 sec)
```

Stored Procedure with Control Flow

```
DELIMITER //

CREATE PROCEDURE GiveBonus(IN emp INT)
BEGIN
    DECLARE emp_salary DECIMAL(10,2);
    SELECT salary INTO emp_salary FROM employees WHERE emp_id = emp;

    IF emp_salary < 50000 THEN
        UPDATE employees SET salary = salary + 5000 WHERE emp_id = emp;
    ELSE
        UPDATE employees SET salary = salary + 2000 WHERE emp_id = emp;
    END IF;
END //
```

```
END //

DELIMITER ;
```

Output:

```
Query OK, 0 rows affected (0.01 sec)
```

```
CALL GiveBonus(1);  -- Ravi Kumar's salary (45000) is below 50000
SELECT emp_name, salary FROM employees WHERE emp_id = 1;
```

Output:

```
Query OK, 1 row affected (0.01 sec)
```

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Ravi Kumar | 50000.00 |
+-----+-----+
1 row in set (0.00 sec)
```

Note: Because Ravi's original salary (45000) was below the 50000 threshold, the IF branch added 5000, bringing it to 50000.00.

Dropping a Procedure

```
DROP PROCEDURE IF EXISTS GetEmployeesByDept;
```

Output:

```
Query OK, 0 rows affected (0.01 sec)
```

Stored Procedures vs Functions

Aspect	Stored Procedure	Function
Return value	May return zero, one, or multiple values (via OUT params)	Must return exactly one value
Usage in SELECT	Cannot be used directly inside a SELECT statement	Can be used inside SELECT/WHERE expressions
Call method	Invoked using CALL	Invoked as part of an expression
Transaction control	Can contain COMMIT/ROLLBACK	Cannot contain transaction control statements

Advantages of Stored Procedures

- Improved performance — precompiled and cached execution plan
- Reduced network traffic — one call executes multiple statements on the server
- Reusability and modularity of business logic
- Enhanced security — users can be given permission to execute a procedure without direct table access
- Easier maintenance — logic centralized in the database rather than duplicated across applications

6.5 Triggers (Related Concept)

A Trigger is a stored program that automatically executes in response to an INSERT, UPDATE, or DELETE event on a specified table.

```

DELIMITER //

CREATE TRIGGER before_salary_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SET NEW.salary = OLD.salary;
    END IF;
END //

DELIMITER ;

```

Output:

```
Query OK, 0 rows affected (0.01 sec)
```

```

-- Ravi Kumar currently has salary = 50000.00 (from the GiveBonus example above)
UPDATE employees SET salary = -500 WHERE emp_id = 1;
SELECT emp_name, salary FROM employees WHERE emp_id = 1;

```

Output:

```

Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

```

```

+-----+-----+
| emp_name | salary |
+-----+-----+
| Ravi Kumar | 50000.00 |
+-----+-----+
1 row in set (0.00 sec)

```

Note: The attempted update to a negative salary was intercepted by the trigger, which reset *NEW.salary* back to *OLD.salary* — so the value stayed at 50000.00 instead of becoming -500.

Quick Revision Summary

Topic	Key Takeaway
DBMS vs RDBMS	RDBMS organizes data into related tables using keys; DBMS is the broader umbrella term
ACID	Atomicity, Consistency, Isolation, Durability — guarantee reliable transactions
Normalization	1NF → atomic values, 2NF → no partial dependency, 3NF → no transitive dependency, BCNF → every determinant is a super key
DDL	CREATE, ALTER, DROP, TRUNCATE — define structure, auto-committed
DML	INSERT, UPDATE, DELETE — manipulate data, can be rolled back
DCL	GRANT, REVOKE — control user permissions
TCL	COMMIT, ROLLBACK, SAVEPOINT — manage transactions
Joins	INNER (matches only), LEFT/RIGHT (all + matches), FULL (all rows), SELF (table to itself)
Views	Virtual table from a stored query, used for simplification & security
Stored Procedures	Precompiled reusable SQL blocks executed with CALL